Science of Computer Programming 252 (2026) 103452

Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico

An agent-Based model reflective architecture for social network sites

Paul Leger [a,*], Daniel San Martín [a], Diego Nuñez [b], Tomás Vélez [b]

[a] EIC, Universidad Católica del Norte, Larrondo 1281, Coquimbo, Región de Coquimbo, Chile
[b] Pragmatics Labs, Universidad Católica del Norte, Larrondo 1281, Coquimbo, Región de Coquimbo, Chile

ARTICLE INFO

Keywords:
Word-of-mouth
Agent-based model
Simulation
Social media
Open implementations
Tower reflection

ABSTRACT

Social Network Sites (SNSs) like Instagram have proven to be effective platforms for Word-Of-Mouth marketing (WOM), and simulations of Agent-Based Models (ABMs) can predict users' behavior in these networks. Using ABMs allows researchers to understand real-world scenarios by modeling and abstracting them, and to extract and identify the essential behaviors to study. To develop an ABM, a programmer can follow two approaches. First, developing a program from scratch that runs a specific ABM simulation. Second, using known frameworks like Repast to speed up implementation. The issue with the first approach is that advanced programming knowledge in ABMs is required, and with the second one, apart from ABM knowledge, a programmer needs a deep understanding of these frameworks. This paper proposes the architecture of a Flexible Agent Simulator for Open WOM (FASOW), which uses a design strategy inspired by the concepts associated with Reflective Tower to provide advanced modularity features and a less steep difficulty curve for developers. This architecture allows developers to decompose a complex model into layers of varying complexity, gradually increasing the required knowledge. To demonstrate the effectiveness of FASOW, we replicated two published ABMs. Additionally, we provide an online, functional proof of concept to showcase the architecture's practical applicability.

1. Introduction

Social Network Sites (SNSs) like Facebook, Instagram, and X (ex-Twitter) allow users to generate contacts and broadcast information among people. These sites currently have millions of active users; for example, Facebook maintains an average of 2.9 billion active users per month Newberry [1], and more than 500 million tweets were shared in 2023 on X [2].

People usually want to read or hear opinions from relatives, friends, or experts about products before buying them [3], in real life and on the internet. This type of communication among groups of people is called Word-Of-Mouth (WOM), which is usually the primary source of information used for its credibility and influence [4]. Hence, companies are interested in using a mechanism based on WOM to spread information that organically influences the perception of some specific products or services [5]. WOM's main benefit is the ease and comfort of sharing information, both for customers and businesses [6–8]. Given this importance, WOM has recently started to be used as a powerful marketing tool: WOM marketing [6]. This type of marketing consists of spreading a message about a particular product or service to a group of customers (*a.k.a. seeds*) to spread the same message further, and information propagation is achieved [9].

* Corresponding author.
E-mail addresses: pleger@ucn.cl (P. Leger), daniel.sanmartin@ucn.cl (D. San Martín), diego.nunez01@alumnos.ucn.cl (D. Nuñez), tomas.velez@alumnos.ucn.cl (T. Vélez).

https://doi.org/10.1016/j.scico.2026.103452
Received 1 January 2025; Received in revised form 2 February 2026; Accepted 5 February 2026
Available online 13 February 2026
0167-6423/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

An Agent-Based Model (ABM) [10,11] is a technique that allows researchers to model and simulate real-world situations by abstracting and identifying only the crucial elements to study a phenomenon. Using this technique, researchers can analyze the behavior and interactions among agents in a determined environment. Agents are individuals who follow a set of (simple) behaviors, have a defined and mutable state, and can interact with other agents. By using ABMs, researchers can model a real-world-like representation of SNSs, where agents can share messages that other agents might read, and where agents can influence the decisions of others. With ABMs, researchers in WOM marketing can test hypothetical scenarios, achieving more success when applying marketing strategies in the real world [11–14].

There are two approaches to implement ABMs. The first is to build an ABM *from scratch* using a programming language and its paradigm, and the second one is to *use frameworks* that are already well-suited for implementing ABMs:

1. **From scratch.** Implementation from scratch using a programming language along with its paradigm; for instance, Object-Oriented Programming (OOP) [15]. This is complex because a considerable amount of abstractions must be designed and programmed to implement the model. The main issue is that experienced software developers are needed, and there will probably be a large number of modules to be implemented from scratch. Finally, this whole process also required a detailed test suite to verify the correct execution of the simulation.
2. **Using ABM frameworks.** Existing frameworks like NetLogo [16], Repast [17], or MASON [18] for simulations, including ABMs, are available. There are benefits of using these frameworks, like reducing the number of modules to be built, having a known protocol to manage the program's output, and having logical support for implementing the models. One of the most used frameworks is currently Repast, which reduces the ABM development efforts by being a general-purpose ABM framework. However, a learning process is needed to use Repast, especially when making deep changes in a model, in which case detailed knowledge is required. Additionally, when working with ABMs in WOM for SNSs, the business logic of SNSs must be implemented from scratch, causing issues like the ones described in the previous point.

As the previous two approaches present issues, we propose FASOW (Flexible Agent Simulator for Open WOM), an architecture that uses the *reflective tower* [19] concept as a design strategy. The reflective tower, which comes from the programming languages, allows developers to organize an architecture with levels, starting from a *base level* to one or more *meta levels*. Using this design strategy, developers can facilitate the modular inclusion of features by incrementally adding or modifying ABM modules in each upper level. The FASOW's architecture is organized in levels representing a reflective tower to implement their ABMs. When the complexity of implementing a feature increases, the reflective tower allows developers to gradually access its upper levels to implement this more complex feature. The gradual implementation of the features enables developers to understand the model granularly by dividing the complex model into difficulty levels. In FASOW, the reflective tower is composed of four levels. The first (and easiest) level is used to configure the input data to run a model; this level can be expressed in a familiar language for ABM researchers in WOM marketing (*e.g.,* HTML interface), preventing requiring extensive software development knowledge. The last (and most complex) level focuses on the core concerns of an agent-based simulation that should not vary across different ABM implementations. We validated the FASOW architecture across three cases on the Twitter site. The first case serves as a baseline for validating our model. The second case is [13], which analyzes the saturation effect on agents when repeatedly interacting with a WOM advertising message. The third use case is [14] to identify the best combinations of agent types to seed a WOM advertisement. Concretely, this version of the paper aims to develop an ABM architecture for WOM in SNSs, using programming-language concepts to facilitate learning and use while implementing ABM models in WOM marketing campaigns. The paper contributions are:

1. **Gradual complexity for developers.** An architecture that allows developers to gradually implement changes to facilitate the learning process of ABMs in WOM marketing for SNSs. To achieve the gradual complexity, the FASOW's architecture design strategy, based on the Reflective Tower concept, allows developers to use four levels of complexity to implement features of an ABM. Through a MetaAPI, developers have access to these levels (Section 3 - Architecture).
2. **For general-purpose languages.** Object abstractions to allow developers to create FASOW in diverse languages that support OOP. Indeed, we provide an online proof of concept, named FASOWts, implemented in TypeScript, a typed version of JavaScript that supports OOP based on classes (Section 3 - Architecture). Additionally, we provide a Web interface to select and configure modules, allowing the creation of an ABM instance.
3. **State-of-the-art/practice.** An extensive and deep reference frame that reflects *a*) the *state-of-the-art* in terms of studies about architecture specifications of ABMs for SNSs and *b*) the *state-of-the-practice* about functional frameworks related to FASOW (Section 4 - Related Work).
4. **Domain-specific architecture.** Considering the previous point, we can claim that FASOW is a pioneering proposal for a domain-specific architecture to implement ABMs in WOM marketing campaigns for SNSs. This architecture enables developers to rapidly implement diverse ABMs by reusing and configuring existing modules across the four domain-specific levels of the architecture (Section 5 - Case Studies).

This journal article is a substantially extended version of our short paper presented at ICEIS 2024 [20]. Beyond the original 8-page version, it now provides a much richer conceptual framework for the Reflective Tower architecture, detailing its levels and scopes, introducing the *metas API* for accessing, adding, and modifying modules at each level, and presenting both static and dynamic views of FASOW, including two new sequence diagrams that show how to (a) build ABMs from existing modules and (b) extend them by adding isolated modules for specific concerns. The validation has been considerably broadened: in addition to the initial saturation-effect scenario on X, we include two new, qualitatively different use cases on optimal seeding strategies for WOM campaigns and

compare them in terms of architectural extensions and metrics. We also incorporate an expanded and rigorous literature review (nine additional pages) that situates our proposal within the state-of-the-art in ABM frameworks and reflective architectures, and contrasts it with current industrial solutions. Furthermore, we now provide full implementation details, including public code availability on GitHub, a citable Zenodo archive (DOI), and an online proof-of-concept to support reproducibility and practical adoption. Finally, this version adds a dedicated section on lessons learned, where we discuss the difficulties encountered, limitations identified, and future work derived from implementing the three case studies with FASOW. For an extensive description of the new in this journal version, refer to A.

The paper is organized as follows. Section 2 presents the concepts required to understand FASOW. Section 3 describes the architecture of FASOW. Section 4 compares our proposal with the state of the art and the state of the practice in frameworks for simulating ABM, for example Repast, Simphony, and Simudyne. Section 5 presents the validations used to examine the capabilities of the architecture. This validation is conducted on one baseline implementation and two pre-existing models, with a focus on WOM marketing campaigns for X. Section 6 discusses the threats to validity. Finally, Section 7 presents the conclusions and current limitations.

Availability. A concrete implementation of FASOW is available on https://github.com/pragmaticslaboratory/fasow (rev. $e54d6ef$) [21], and an online proof of concept of our proposal is on https://sasow-org.github.io/fasow-monorepo [22]. The DOI of the running version of FASOW is on Zenodo: https://zenodo.org/records/15817106.

2. Background: WOM in SNSs and reflective tower

This section introduces the main concepts required to contextualize our proposal. First, Word of Mouth (WOM) marketing campaigns in social network sites are outlined. Second, agent based models and their use in WOM marketing are briefly reviewed. Finally, the reflective tower concept from programming languages is introduced as a foundation for the proposed architecture.

2.1. WOM Based marketing campaigns

Marketing research indicates that awareness of products or services is strongly influenced by the dissemination of information [23]. As a result, Word of Mouth (WOM), defined as information exchange among individuals, has become a relevant mechanism for companies [6]. Its electronic counterpart, known as electronic WOM (eWOM), corresponds to opinions disseminated through online platforms [4,24]. The increasing availability of online communication tools has amplified the impact of eWOM on consumer decision making [25].

Social Network Sites (SNSs) provide a natural medium for eWOM, allowing users to publish opinions and interact without direct personal contact. Through these interactions, consumers are able to form perceptions and make purchasing decisions based on information shared by peers [24,26]. In particular, opinions originating from individuals with prior experience are often perceived as highly influential [26].

When consumers are encouraged by a company to disseminate information, the process is referred to as WOM marketing [24]. Such campaigns typically start from a limited set of initial consumers, called *seeds*, whose characteristics enable effective information propagation [14]. Previous studies have analyzed the role of heterogeneous user types in WOM dynamics [13,14,27–29], including highly connected users (*hubs*) [27] and combinations of distinct consumer roles influencing dissemination speed and reach [28,30].

An SNS can be abstracted as a directed or bidirected graph of users exchanging information according to platform specific interaction mechanisms (e.g., “share” or “like”). The objective of a WOM campaign is to define agent characteristics and message properties that maximize information diffusion over this network, while poorly designed campaigns result in limited dissemination.

2.2. Agent based model in WOM marketing campaigns

An Agent Based Model (ABM) [10,11] is a simulation paradigm in which the behavior of a population emerges from interactions among individual agents and their environment [31]. Agents evolve over discrete time steps, following local rules that collectively give rise to complex system dynamics. Compared to aggregate modeling approaches, ABMs are well suited to represent heterogeneous actors, localized interactions, and emergent phenomena.

ABMs offer several advantages for the analysis of social and marketing systems: they enable complex behavior to be generated from simple rules, support experimentation under limited data availability, facilitate the exploration of hypothetical scenarios, and reduce modeling costs by emphasizing theoretical consistency over large scale empirical datasets 1.

An ABM for WOM campaigns can be described by a 4 tuple $(Q_0, \Sigma, \alpha, \Gamma)$, whose components are modularly defined as follows:

1. **Calibration (Q_0)** specifies the initial state of the model, derived from theoretical assumptions, empirical data, or a combination of both.
2. **Environment (Σ)** represents the interaction space, which may correspond to an online network (e.g., an SNS) or an offline setting [32].
3. **Agents (α)** denote individuals or entities endowed with internal states and interaction capabilities [10].
4. **Actions (Γ)** define the set of rules governing agent behavior and state transitions over time [27].

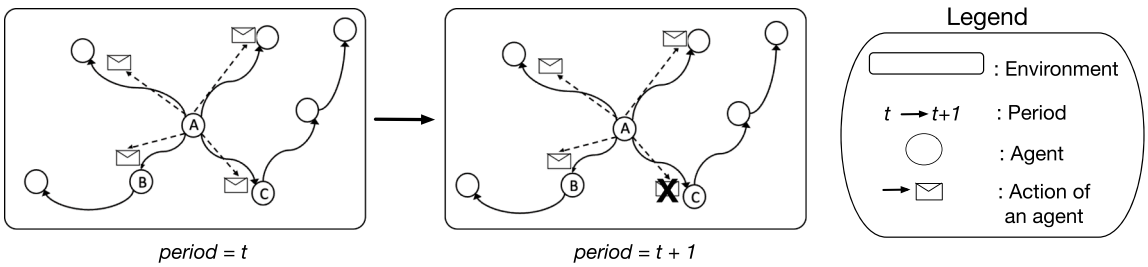


Fig. 1. Illustrative ABM simulation over two consecutive periods ($t \rightarrow t+1$).

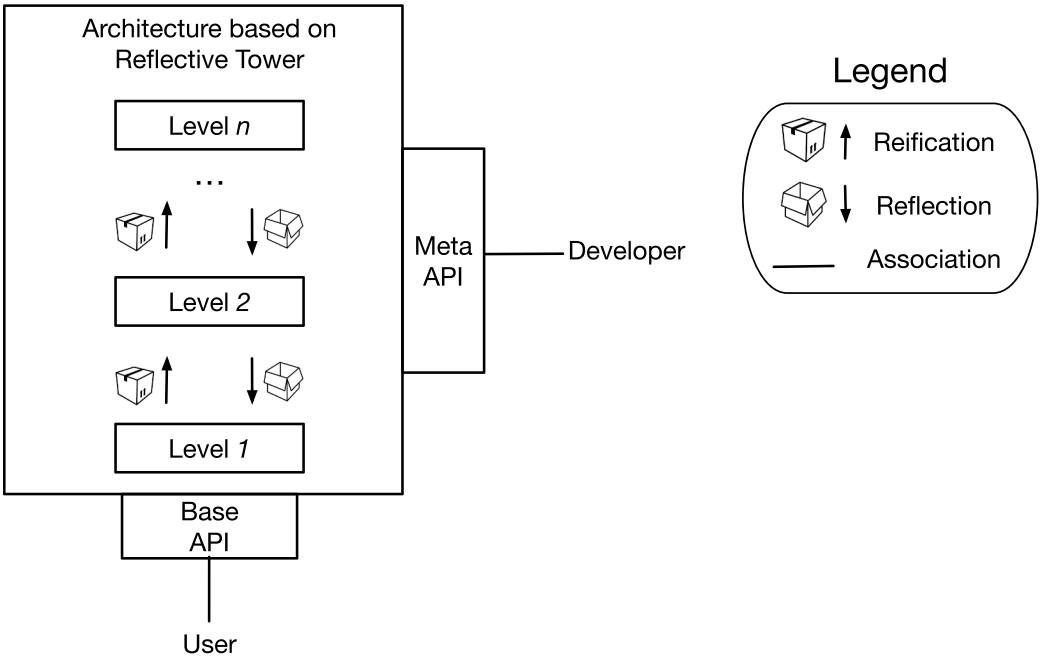


Fig. 2. Architecture based on a reflective tower.

ABMs have been shown to be particularly effective for modeling WOM marketing campaigns, as they capture interaction driven diffusion processes using relatively small amounts of data for calibration and validation [32,33]. A growing body of work has applied ABMs to study information diffusion and marketing strategies in SNSs [12,14,34–36]. Although their adoption in marketing research is comparatively recent [37], ABMs are increasingly recognized as a suitable methodology for analyzing the diffusion of products and services in online social environments [13,14,27,32,38,39].

2.3. Reflective tower

In computer science, a *reflective system* is capable of representing and manipulating aspects of its own execution [19]. A *reflective tower* structures such a system into multiple levels, where each level provides introspective access to the behavior of the level below. This concept originates in programming languages research and has been explored in languages such as Lisp, Java, and Smalltalk [40–42].

While conventional object oriented mechanisms (e.g., inheritance or polymorphism) support the definition of families of classes, they typically do not expose execution concerns such as scheduling, instantiation, or data collection as configurable elements. A reflective tower addresses this limitation by separating base level behavior from meta level mechanisms that can inspect, configure, and extend that behavior.

In the context of agent based modeling, this approach enables the internal structure of an ABM, namely calibration, environment, agents, and actions, to be explicitly accessed and extended through well defined meta level interfaces, rather than being embedded in monolithic implementations. Reflection is commonly described through the dual processes of *reification*, which lifts execution details to a higher level, and *reflection*, which applies meta level modifications back to the base level [19].

Fig. 2 illustrates a reflective architecture distinguishing between user facing base interfaces and developer oriented meta interfaces. The base interface provides access to core functionality, while the meta interface supports controlled extension and customization of the underlying system.

Within this architectural style, the reflective tower can be interpreted as a specialized layered architecture, where each level encapsulates a coherent concern and exposes constrained interfaces to adjacent levels. In this work, the approach is instantiated for WOM oriented ABMs using four levels, Calibration, Environment, Agent, and Action, each providing both base functionality and meta level extension points.

3. FASOW: A reflective architecture for ABMs in SNSs

This section introduces our proposal: the *Flexible Agent Simulator for Open WOM* (FASOW), a reflective architecture for simulating ABMs in WOM marketing over Social Network Sites (SNSs). FASOW builds upon the reflective tower (Section 2.3), which structures an ABM into progressively richer layers of abstraction. This organization enables the controlled extension of behavior, allowing developers to introduce new features without altering the underlying architecture.

The remainder of this section is organized as follows. We first present the static and dynamic design of the architecture. We then describe FASOWts, a concrete implementation written in TypeScript, for which a proof-of-concept is publicly available at https://sasow-org.github.io/fasow-monorepo [22]. Finally, we summarize the main benefits of our proposal.

3.1. Static design of the architecture

FASOW adopts a layered architecture organized as a reflective tower, where each level encapsulates a specific function of an ABM and exposes clearly defined input/output service points. The `Calibration` level receives experiment specifications and produces an instantiated model; the `Environment` level encapsulates SNS dynamics and exposes operations to query and update network state; the `Agent` level defines agent types and their state transitions; and the `Action` level determines how actions are composed and applied over time. Communication between these levels is mediated by three facades, `BaseAPI`, `TowerHandler`, and `DataHandler`, which implement the top-down and bottom-up flows: users submit configuration requests to `BaseAPI` (forwarded to `Calibration`); developers register new modules through `TowerHandler`; and simulation outputs are collected and returned via `DataHandler`. This encapsulation and controlled flow follow the classical principles of layer-oriented architecture.

Fig. 3 presents the static design of FASOW using a C4 container-level view of the system´s main architectural building blocks. Following the reflective-tower structure described in Section 2.3, FASOW organizes the concerns of an ABM in WOM marketing for SNSs into four incremental *levels*: `Calibration`, `Environment`, `Agent`, and `Action`. Each level corresponds to one of the conceptual ABM components introduced in Section 2.2, providing a structured and domain-specific separation of responsibilities ranging from model instantiation to environment behavior, agent representation, and action composition, mirroring the conceptual components introduced in Section 2.2.

At the container level, FASOW exposes three essential modules that orchestrate the four levels of the tower:

- **`BaseAPI`.** A user-facing API that receives the input parameters of a simulation and delegates them to the `Calibration` level. This module enables users to configure experiments without requiring knowledge of the internal architecture.
- **`TowerHandler`.** A facade that provides unified access to the four levels of the reflective tower. It registers new modules, manages their configuration, and allows developers to navigate increasingly specialized abstractions.
- **`DataHandler`.** A module that defines and collects the output produced by a simulation. Because different ABMs may require distinct output structures, this handler modularizes and centralizes the definition of output data independently from the core simulation logic.

Below these handlers, the four levels represent the domain-specific structure of the reflective tower:

- **`Calibration Level`.** Gathers user parameters, selects the modules required for a specific ABM instance through the handlers, and assembles the resulting configuration.
- **`Environment Level`.** Defines the behavior of the SNS in which agents interact.
- **`Agent Level`.** Specifies the different types of agents, their properties, and their configuration.
- **`Action Level`.** Encapsulates the actions that agents can perform and the rules that describe how these actions are combined.

The levels are interconnected according to the reflective-tower structure. During the construction of a concrete ABM instance, the `Calibration` level invokes the `Environment`, `Agent`, and `Action` levels, which provide increasingly specialized abstractions. Although `Calibration` is the entry point for user-defined configuration, it depends on the abstractions offered by the upper levels to assemble the full model. Feedback may flow in the opposite direction, but these interactions do not introduce architectural dependencies, thereby preserving the hierarchical organization of the tower.

To describe the internal composition of each level, Fig. 4 presents a C4 component-level view of a generic level X. Every level contains the same four modules:

- **`MetaAPI`.** A facade exposing extension points and connecting the level with `TowerHandler` and `DataHandler`.
- **`MetaLevel`.** Implements reflective mechanisms to configure the core logic of the level and manage its extensions.

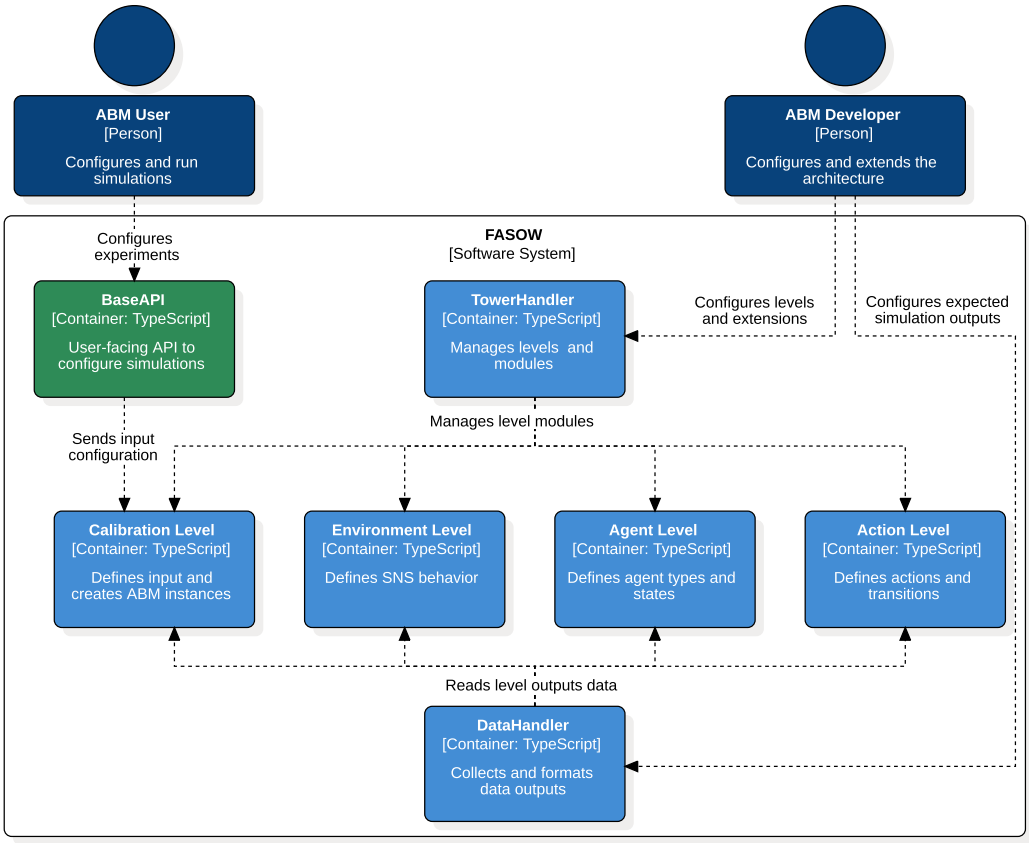


Fig. 3. C4 container-level view of FASOW, showing the four levels of the reflective tower and the coordinating modules (`BaseAPI`, `TowerHandler`, and `DataHandler`).

- `Level`. The core module containing the main logic of the level, such as SNS behavior, agent definitions, or action composition.
- `LevelExtensions`. A set of developer-defined modules that extend the level´s behaviour. New features for ABM variants are incorporated here.

At the implementation level, each tower level is realized as a compact object-oriented hierarchy that enables both abstraction and specialization. The `Level` class defines the generic behaviour of the layer (for example, the abstract notion of an "agent", "environment" or "action"), while `LevelExtensions` contains concrete subclasses that implement specialized variants such as particular agent types or SNS-specific environment rules. The `MetaAPI` provides the interfaces for registering, configuring, and validating these subclasses, whereas `MetaLevel` coordinates their lifecycle within the tower. This class schema makes explicit how developers reuse uniform abstract interfaces while adding tailored specializations for distinct ABM scenarios, reinforcing the modularity and extensibility of the reflective architecture.

It is important to note that Fig. 4 instantiates this generic structure using the `Calibration` level as an illustrative example. Among all levels of the reflective tower, `Calibration` is the only one that interacts directly with the global `BaseAPI`. This is because all user-defined simulation parameters enter the architecture through `Calibration`, which is solely responsible for assembling a complete ABM configuration from user input.

While the discussion above described `BaseAPI` and `MetaAPI` at the architectural (container) level, the component-level view of Fig. 4 clarifies how these APIs operate *within* each tower level. For `Calibration`, the request received by `BaseAPI` is delegated directly to its core `Level` module, which collects the required modules through `TowerHandler` and prepares the configuration that will be executed.

The resulting structured simulation output is then assembled and returned by `DataHandler`, which centralizes both the definition and collection of output across all levels. For the remaining levels (`Environment`, `Agent`, and `Action`), no such direct request/response with `BaseAPI` takes place. Instead, all configuration and extension operations are mediated exclusively through each level´s `MetaAPI`, which receives developer-facing instructions via `TowerHandler`.Such operations may register new `LevelExtensions`, adjust existing behaviors, or expose configuration parameters that later surface in `Calibration` as user-selectable options through the `BaseAPI`.

This asymmetry is intentional: `Calibration` embodies the user-facing entry point of the architecture, while the deeper domain-specific levels are accessible only through meta-level operations. As a result, the reflective tower preserves a clear separation between

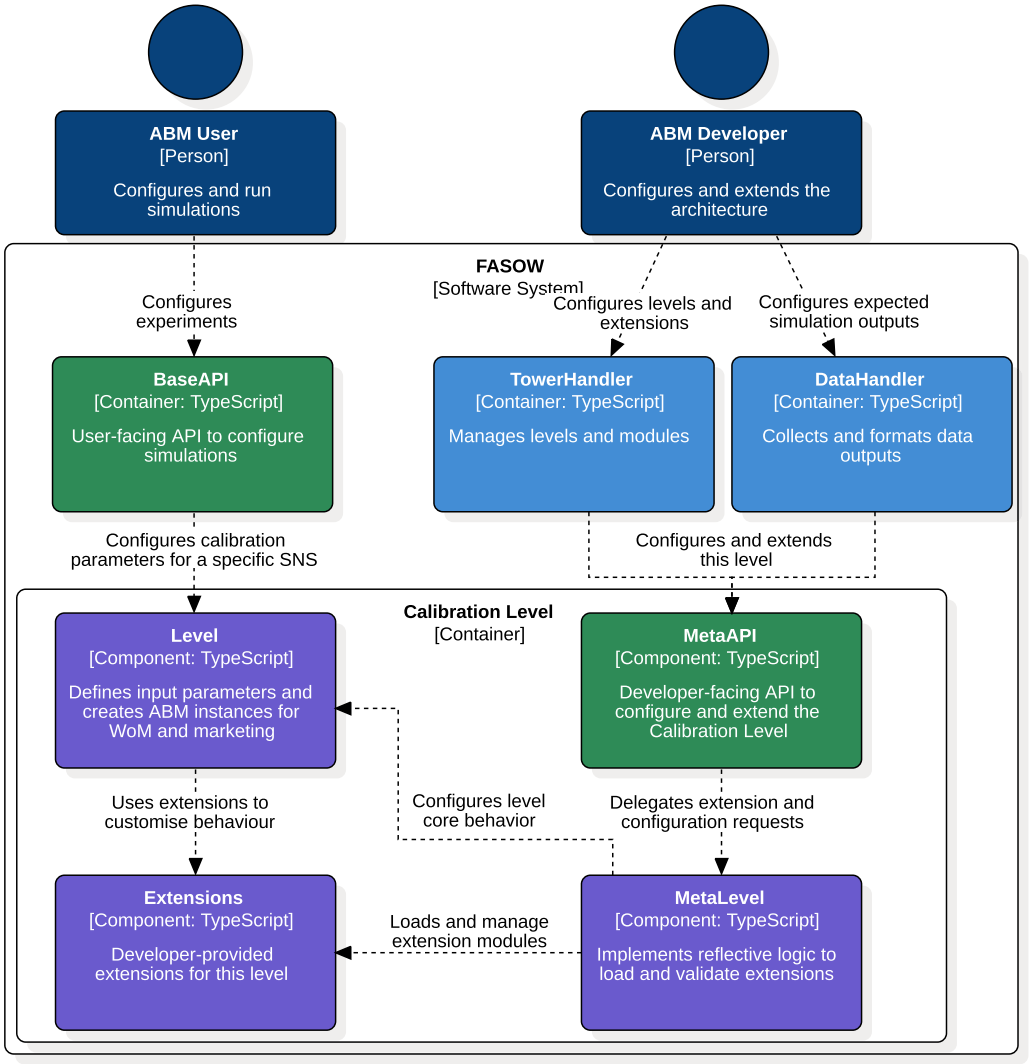


Fig. 4. C4 component-level view of a generic level X in the reflective tower, composed of `MetaAPI`, `MetaLevel`, `Level`, and `LevelExtensions`.

user-level configuration and developer-level extension, ensuring that only the first layer exposes service endpoints to end users, whereas internal abstractions remain encapsulated behind reflective mechanisms.

3.2. Dynamic design of the architecture

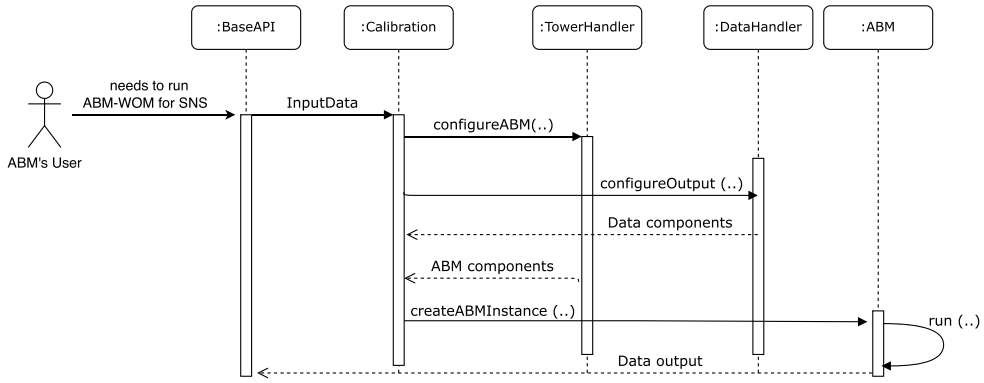
While the static design describes the architectural structure of FASOW, the dynamic design focuses on the runtime interaction among system modules. Fig. 5 presents two sequence diagrams that capture the main interaction patterns supported by the architecture: the creation and execution of an ABM instance using existing modules, and the extension of the architecture through the addition of new functionality.

3.2.1. Creating and extending an ABM

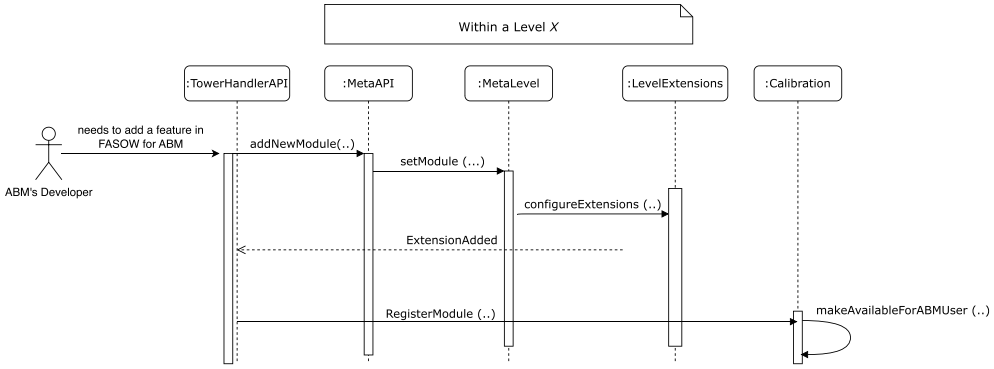
Two complementary interaction scenarios are supported by the architecture.

First, when an ABM instance is created and executed through the Base API, configuration parameters required for the simulation are provided by the user. These parameters are processed by the `Calibration` level, which coordinates with `TowerHandler` and `DataHandler` to retrieve the required modules, including environment behavior, agent definitions, action rules, and output specifications. Once the configuration is assembled, execution of the ABM instance is initiated. During this process, only the `Calibration` level is exposed to the user, while the remaining architectural levels remain encapsulated.

Second, architectural extensions are introduced through the Meta API. In this case, interactions are mediated by `TowerHandler`, which communicates with the `MetaAPI` of the corresponding level. This mechanism enables new modules to be added or existing



(a) Creation and execution of an ABM instance through the Base API



(b) Extension of the architecture through the Meta API

Fig. 5. Sequence diagrams illustrating the interaction patterns supported by the Base and Meta APIs.

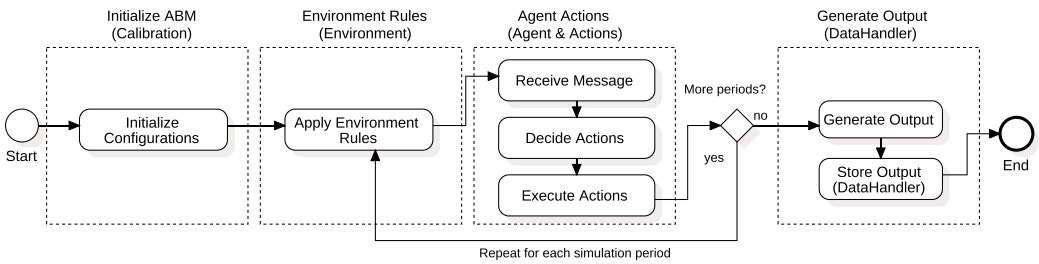


Fig. 6. Execution flow of an ABM organized into four conceptual phases.

ones to be modified without exposing internal implementation details. After registration, the new functionality becomes available for configuration through the `Calibration` level. This interaction path is intended for developers and preserves the simplicity of the Base API for end users.

3.2.2. Running an ABM

Once an ABM instance has been assembled, execution proceeds according to the process illustrated in Fig. 6, which is organized into four conceptual phases. First, the simulation is initialized by the `Calibration` level using user provided parameters such as the number of agents, the selected SNS, the simulation duration, and the initial agent types.

Next, environment rules associated with the selected SNS are applied to govern message dissemination and propagation. During each simulation period, agents receive messages, decide on appropriate actions, and execute those actions according to the defined interaction rules. After the execution of agent actions, a conceptual decision point determines whether additional simulation periods remain to be executed. If so, the process iterates by reapplying the environment rules; otherwise, execution proceeds to the final phase.

In the final phase, output data are generated and stored according to the output model defined by the `DataHandler`. This execution flow is consistent with the static architecture. Configuration is initiated through the `BaseAPI`, user input is mediated by the

```
class XAgent extends Agent {
  step(): void {  this.receiveMessage();
  }

  createAgent(id: number, config: MetaAgentConfig): Agent {
    return new XAgent(id, config);
  }
}

class AgentAPI {
  private factories = new Map<string, typeof Agent>();

  // MetaAPI: register a new agent extension
  registerAgent(type: typeof Agent): void {
    this.factories.set(type.name, type);
  }

  // Instantiate agents from registered extensions
  createAgent(typeName: string, id: number, config: MetaAgentConfig): Agent {
    const { createAgent } = Reflect.construct(
      this.factories.get(typeName)!, [],
    );
    return createAgent(id, config);
  }
}
```

Listing 1. Specialized agent type and its instantiation via the Agent MetaAPI in FASOWts.

`Calibration` level, lower architectural levels provide increasingly specialized functionality, and coordination of execution and data collection is maintained throughout the simulation lifecycle.

3.3. FASOWts

Following the FASOW architecture presented in Fig. 3, we develop a running version of our proposal: FASOWts [21]. This version is a concrete implementation in TypeScript, a typed version of JavaScript that supports abstractions like classes and interfaces[1] In FASOWts, developers use the `BaseAPI` to configure and create an ABM instance. To add extensions that satisfy specific new ABM requirements, FASOW's `MetaAPI` provides a set of interfaces to access and configure at each level in the tower. Each level also offers interfaces to allow developers to add `LevelExtensions` (*e.g.,* new types of agents).

Listing 1 provides a concrete illustration of how a tower level is implemented in FASOWts. Here, `XAgent` specializes the abstract behaviour defined by the `Agent` class and represents a `LevelExtensions` module. The corresponding `AgentAPI` serves as the level´s `MetaAPI`, registering available agent types and instantiating them through a factory method `createAgent`. This example makes explicit how abstraction (`Agent`) and specialization (`LevelExtensions`) are separated and coordinated within each level of the reflective tower.

To test FASOWts, we implement it as a back-end service, and we provide a Graphical User Interface (GUI) using a Web interface [22]. Fig. 7 shows a screenshot of this Web interface, which works as `BaseAPI` and does not require advanced knowledge of FASOW.

This GUI enables developers to configure and create an ABM instance, in particular *a*) select modules, for example specific agent types, from each level, and *b*) configure the selected modules. In addition, the interface offers a friendly entry point for users who do not have expertise in software development or simulation engineering, since it allows them to configure experiments without interacting directly with the underlying code. For more advanced scenarios, where the creation of new agent behaviors or environment rules is required, developers can rely on the programmatic interfaces provided by the architecture. In this way, the Web interface supports accessible experimentation for domain specialists while the API ensures full extensibility for developers.

3.4. Benefits

This section introduced the FASOW architecture from both static and dynamic perspectives, using C4 container and component diagrams as well as sequence and process views. The reflective-tower design brings several benefits for both users and developers:

1. **Gradual complexity for developers.** The reflective tower organizes ABM concerns hierarchically, enabling developers to navigate the architecture in increasing levels of abstraction. New modules can be added incrementally through level-specific extension mechanisms, without requiring modifications to existing components. This reduces cognitive load and simplifies maintenance.

[1] TypeScript works on Nodejs [43], a JavaScript runtime environment outside a browser (*e.g.,* servers), through a compilation process.

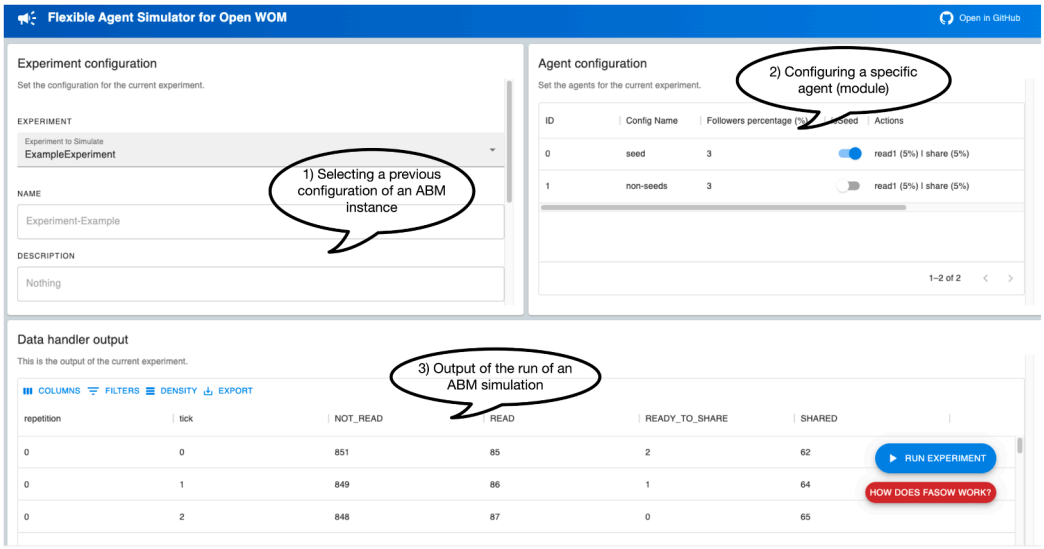


Fig. 7. The client Web interface, a proof-of-concept, of FASOWts. First, select or create a configuration of an ABM instance. Second, configure a specific module. Third, after a run, visualize the results.

2. **Domain-specific architecture.** Unlike general-purpose ABM frameworks, FASOW is tailored to WOM marketing over SNSs. It maps the four conceptual components of an ABM (calibration, environment, agents, and actions) to explicit architectural levels, offering a structured and domain-relevant foundation for constructing ABM experiments in this context.
3. **Language independence.** The architecture is expressed using standard Object-Oriented Programming (OOP) concepts and design patterns [44], without relying on language-specific features. Although FASOWts provides a TypeScript implementation, the reflective tower can be implemented in any general-purpose, object-oriented language.
4. **Benefits for users and developers.** From a *user's* perspective, FASOW exposes only the `BaseAPI`, allowing non-technical users, such as marketers or domain experts, to configure and execute ABM experiments without interacting with internal architectural components. From a *developer's* perspective, the reflective tower offers a modular and extensible mechanism for introducing new behaviors, agent types, environment rules, or output definitions through the `MetaAPI` of each level. This clear separation between user-facing configuration and developer-facing meta-level operations enables incremental extension without modifying existing code, facilitating collaboration between technical and non-technical stakeholders.

4. Related work

This section delves into research on tools and methodologies for architecting Agent-Based Models (ABMs). While our primary focus is on studies that have developed custom approaches closely aligned with our work, we also spotlight generic methods that have significantly contributed to this field. The findings are organized into two sections. The first summarizes papers that have advanced the state-of-the-art in using ABMs to model behaviors across various domains. The second offers a state-of-the-practice overview, presenting a list of generic tools and approaches for ABM modeling. We emphasize the expressiveness and extensibility of these tools and approaches, as these dimensions are critical for comparison with our proposal.

4.1. State-of-the-art in ABMs for WOM

Although ABM has been applied for decades to study consumer behavior, its use in simulating WOM dynamics remains comparatively limited. ABM is particularly well-suited for capturing the heterogeneity of consumers and the emergent patterns that arise from their interactions. Yet, only a modest number of studies have directly explored WOM processes using this approach.

In this section, we briefly review key contributions that align with the scope of our work. These studies use ABM to examine various aspects of consumer decision-making, marketing interventions, and information diffusion, with some explicitly modeling WOM mechanisms.

Table 4.1 summarizes the reviewed literature, outlining their simulation environments, modeling goals, agent structures, input/output variables, and software platforms. This synthesis provides a foundation for situating our approach, which contributes a configurable and extensible ABM designed specifically for WOM simulations in social networking environments.

Amini et al. [38] Presents an ABM for new product diffusion as a complex adaptive system influenced by consumer interactions and WOM. It models autonomous consumer agents and explores various sales policies. *Limitation:* Assumes consumer homogeneity and fixed marketing parameters.

Table 1
ABM scenarios for WOM, ordered by year.

Ref.	Dimension	Description	Software
[38], 2012	Environment	X	Fortran
	Objective	Study SC strategies on profit during diffusion	
	Agents	Consumers	
	Input	Behavior, market, network	
	Output	NPV and adoption rate	
[45], 2016	Environment	Virtual market	NetLogo
	Objective	Analyze customer behavior and online decisions	
	Agents	Consumers	
	Input	Gender, age, income, sensitivities	
	Output	Market shares, visits, strategies	
[13], 2017	Environment	X	JS web app
	Objective	Analyze message repetition in tweets	
	Agents	X users (avg. 208 neighbors)	
	Input	Tweet repetition, followers, analytics	
	Output	Reads, wear-out, retweets	
[46], 2017	Environment	Unspecified	Unspecified
	Objective	Optimize WOM strategies and forecast adoptions	
	Agents	Individual or grouped customers	
	Input	Initial state, KPIs, seasonality	
	Output	Adoption time series	
[14], 2019	Environment	X	Java
	Objective	Study avg. users, hubs, leaders in WOM	
	Agents	Consumers, hubs, opinion leaders	
	Input	Agent types and proportions	
	Output	Diffusion rate, engagement	
[47], 2020	Environment	Weibo	NetLogo
	Objective	Impact of pos/neg agents on brand/WOM	
	Agents	Positive/negative communicators	
	Input	Types, numbers, evolution params	
	Output	Info spread, brand reputation	
[12], 2022	Environment	Cross-border e-commerce	Java
	Objective	Forecast future markets for decisions	
	Agents	Buyers and B2C marketplaces	
	Input	Preferences, attributes, survey data	
	Output	Sales per marketplace	
[48], 2023	Environment	Unspecified	NetLogo
	Objective	Assess eWOM strategies in hospitality	
	Agents	Customers, businesses	
	Input	Customer, business, context, ads	
	Output	Volume, behavior, performance	
[49], 2024	Environment	Amazon + China e-commerce	Java
	Objective	Study market share and oligopoly	
	Agents	Buyers, Amazon, marketplaces	
	Input	Preferences, attributes, surveys	
	Output	Sales per marketplace	

Marković et al. [45] Integrates ABM into innovation management, emphasizing adaptable software architectures to simulate market and organizational dynamics, including supply chains. *Limitation:* Lacks concrete implementation of enhanced simulation tools.

López et al. [13] Uses ABM to study message repetition in Twitter (X) campaigns. Finds that optimal tweet repetition is up to three before audience fatigue. *Limitation:* Focused only on Twitter; possible data biases.

Chica and Rand [46] Proposes an ABM-based decision support system for WOM marketing in freemium apps, supporting stakeholder engagement and strategic evaluation of WOM dynamics. *Limitation:* Implementation details and generalizability are not deeply explored.

Araya et al. [14] Simulates information diffusion on Twitter (X), comparing average users, hubs, and opinion leaders. Finds opinion leaders most effective in spreading WOM. *Limitation:* Platform-specific; no predictive component.

Liao et al. [47] Examines eWOM dynamics in social media via ABM, focusing on how social networks shape consumer decisions. *Limitation:* Less emphasis on actionable strategies or platform diversity.

Table 2
Expressivity/Extensibility Rating Definitions.

Rating	Definition
1	Poor: Very limited expressivity/extensibility, minimal customization options.
2	Fair: Some expressivity/extensibility, but significant limitations exist.
3	Good: Reasonable expressivity/extensibility with moderate flexibility.
4	Excellent: Extensive customization options and high flexibility.

Terán et al. [12] Develops an ABM for Chinese cross-border e-commerce, with agents representing buyers and marketplaces. Captures emergent buyer behaviors and preference adaptation. *Limitation:* Marketplaces do not adapt; lacks predictive capabilities.

Zhang et al. [48] Assesses generic WOM strategies through ABM. Enables testing of scenarios before implementation. *Limitation:* General-purpose model; limited contextual specificity.

Terán et al. [49] Analyzes oligopolistic dynamics between Chinese marketplaces and Amazon using ABM, extending the 2022 study. Buyers adapt, but marketplaces do not. *Limitation:* Same as previous work, no marketplace adaptation or forecasting.

Agent-based models (ABMs) are widely used to simulate complex systems in domains such as social networks, supply chains, and tourism. These models involve diverse types of agents, e.g., users in SNSs or stakeholders in supply chains, interacting within varied environments.

Simulation contexts range from social platforms like X, used to study information diffusion, to marketplaces and tourism systems focused on optimizing routes or product flows. Tools like NetLogo are commonly employed due to their ability to model parallel agent interactions and capture emergent behaviors.

Inputs typically include survey data and behavioral parameters, while outputs reveal dynamic patterns and support strategic decision-making.

4.2. State-of-the-practice in ABMs for WOM

This subsection presents an overview of various tools and approaches used in ABMs for simulation purposes. We concentrate on general-purpose approaches, as our methodology can be categorized within this domain. By exploring widely adopted tools, we aim to highlight the *expressivity* and *extensibility* concerns that general-purpose ABM solutions.

The comparison we conducted forms the basis for evaluating how our approach compares to existing proposals, particularly in terms of flexibility for modeling complex systems, as well as integration and customization capabilities. Expressivity and extensibility are rated based on a consensus among the authors, using the quality ratings outlined in Table 4.2.

By *expressivity*, in the context of programming languages and modeling frameworks, we refer to the ability of a system to represent a wide range of computational constructs and behavioral patterns. In programming-language theory, expressivity relates to the extent to which a language can encode abstractions, control-flow mechanisms, data structures, or concurrency models without resorting to cumbersome encodings [50,51]. Translating this notion to Agent-Based Models (ABMs), expressivity denotes the flexibility with which a platform enables modelers to define agent types, interaction rules, environment dynamics, emergent behaviors, and domain-specific metrics [17,52]. An expressive ABM platform supports a broad variety of modeling paradigms, allowing researchers to represent complex, heterogeneous systems with minimal restrictions and to tailor simulations to specific research questions.

By *extensibility*, we refer to the ease with which an architecture or framework can be modified, expanded, or integrated with new components and tools. Extensibility is widely recognized as a key property in software architecture, reflecting the ability of a system to evolve without compromising its internal integrity [53,54]. In the context of ABMs, extensibility includes adding new agent behaviors, integrating external data sources, incorporating domain-specific modules, or extending output-processing capabilities [55]. An extensible platform allows developers to incrementally introduce new features, ensuring long-term adaptability as models grow in complexity. This is particularly important in research domains where model structures must evolve to accommodate new hypotheses, data, or modeling requirements.

Although these dimensions are widely used to characterize ABM tools, they necessarily involve a degree of subjective judgment. To mitigate this, our assessment in Table 3 draws upon published documentation, case studies, and comparative analyses in the ABM literature, but should still be interpreted as indicative rather than absolute. The table summarizes a variety of ABM platforms according to licensing, implementation languages, and their relative expressivity and extensibility, following the definition of these terms above.

A wide range of agent-based modeling (ABM) platforms is available, each with varying degrees of expressiveness and extensibility. Commercial tools such as AnyLogic [63], ExtendSim [64], FlexSim [65], and Symudine [71] offer robust modeling capabilities, though they are generally less extensible than open-source alternatives. Open-source tools–including NetLogo [16], Repast [70], TerraME [59], Simona [67], and MATSim [69]–provide high flexibility and extensibility, making them well-suited for a broad range of simulations. Specialized platforms such as BDI4Jade [60] and JaCaMo [66] support advanced reasoning via the BDI architecture, while others like ReMobidyc [62] and Sim2APL [61] offer moderate extensibility tailored to specific domains.

Table 3
Summary of Software Agent Tools/Platforms, ordered by the last year of update.

Tool	Lic	Lan	Exp	Ext	Justification
EcoLab(2010) [56]	O	Java, Groovy	4	4	Highly flexible for ecological simulations; highly extensible.
TuCSoN(2015) [57]	O	Java	4	3	Allows for complex agent interactions; good extensibility.
SocLab(2016) [58]	O	Java	4	3	Supports complex social interactions; good extensibility.
TerraME(2020) [59]	O	Lua	4	4	Highly expressive for ecological modeling; easily extensible.
BDI4Jade (2021) [60]	O	Java	4	4	Highly flexible for defining agent behaviors; easily extensible.
Sim2APL(2021) [61]	O	Java	3	3	Provides reasonable flexibility; moderate extensibility.
ReMobidyc(2023) [62]	O	Smaltalk	3	2	Moderate expressiveness; limited extensibility options.
NetLogo(2023) [16]	O	StarLogo	4	4	Highly flexible for defining agents and interactions; easily extensible.
AnyLogic(2024) [63]	C	Java	4	3	Supports multimethod modeling; extensibility is good but somewhat limited.
ExtendSim(2024) [64]	C	C++	3	3	Offers various modeling approaches; moderate extensibility.
FlexSim(2024) [65]	C	C++	3	2	Allows for some customization; limited extensibility.
JaCaMo(2024) [66]	O	Jason	4	4	Modular architecture allows extensive customization; highly extensible.
Simona(2024) [67]	O	Scala	4	4	Supports complex models with custom agent behaviors; highly extensible.
FLAME(2024) [68]	O	CUDA, Python	4	3	Supports complex simulations; extensibility is good but not extensive.
MATSim(2024) [69]	O	Java	4	4	Highly expressive for large-scale transport simulations; modular and easily extensible.
Repast(2024) [70]	O	Java, Groovy	4	4	Highly extensible and expressive for complex modeling.
Symudine(2024) [68]	C	Java	4	3	Highly expressive for complex agent-based models; good extensibility.
FASOWts(2024) [21]	O	TS	4	4	Highly flexible to define ABMs in WOM marketing for SNSs.

* Commercial (C), Open Source (O)

Our proposed platform, *FASOWts* [21], focuses specifically on simulating WOM marketing campaigns in social networking sites (SNSs). Unlike general-purpose ABM tools, it provides predefined models and behavior templates, reducing implementation overhead for developers. Its flexible architecture supports gradual customization at multiple abstraction levels, and, similar to FLAME GPU [68], it uses decorators to facilitate the generation of experiment outputs without requiring extensive standardized or repetitive configuration code.

5. Case studies: Assessment through replication

Replication in ABM refers to the process of reimplementing an existing model, referred to as the original model, validating its findings, assessing its robustness, or exploring its applicability in new contexts. The resulting implementation, known as the replicated model, seeks to reproduce the behavior and outcomes of the original model while introducing variations in specific dimensions such as time, hardware, programming language, toolkits, algorithms, and authors [72,73]. Our replication assessment of three case studies: one that works as a baseline, and two from prior literature to evaluate the capacity of FASOW's architecture to emulate other ABMs. These cases serve as benchmarks for assessing model replication fidelity, functional coverage, and flexibility in introducing new features.

The first case study simulates an electronic WOM marketing campaign that uses a brand's followers to disseminate news about a product. The remaining two case studies are based on the analysis presented in Section 4.1. We selected two use cases to replicate and assess the effectiveness of the FASOW architecture, as illustrated in Fig. 3, in reproducing Agent-Based Models (ABMs) from prior studies. The second case replicates the findings of [14], demonstrating how different combinations of seed agents enhance message diffusion. This replication required only two extensions in FASOW. For the third case study, we replicated the ABM from López et al. [13], which investigates how repetitive messaging in a WOM marketing campaign can overwhelm agents. This replication involved extending each level of FASOW's reflective tower. Details on these use cases and their replication are provided in Table 4.1.

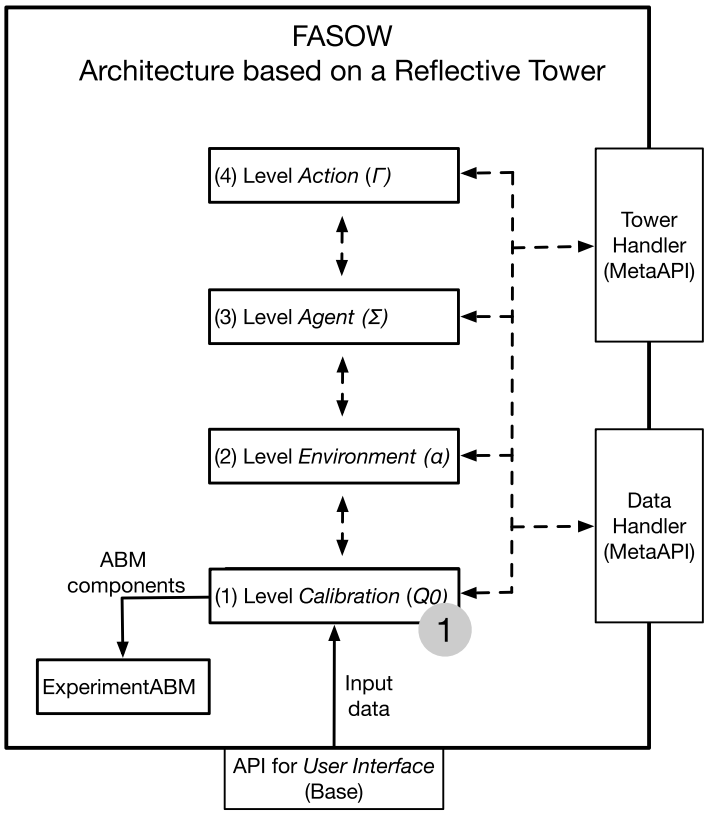
5.1. Case study 1: WOM marketing campaign through sending messages

5.1.0.1. ***Case Overview.*** This case study serves as a baseline for the subsequent two case studies, enabling us to compare the three case studies using quantitative metrics: Afferent Coupling (Ca), Efferent Coupling (Ce), and Instability (I). The replication was conducted using the FASOW architecture to assess its ability to support modular configuration of agents that read and retweet a message with a probability. The target platform is X, where WOM marketing strategies are crucial to real-world campaign outcomes.

5.1.0.2. ***Objectives.*** This case replicates the Agent-Based Model (ABM) that aims to count the number of read messages when a set of seeds receives a message from a brand.

5.1.0.3. ***Data Description.*** Two different agent types are modeled:

- **Seeds**: Fewer agents that receive messages from the brand about the launch of a product or service.
- **Followers**: Many agents that can follow seeds and/or other agents.



Case study:
WOM marketing campaign through sending messages

Fig. 8. Using FASOW to simulate WOM marketing campaign through sending messages, where only the `Calibration` level is necessary.

5.1.0.4. ***Method Application.*** To replicate the ABM, only one key configuration is introduced in the FASOW reflective tower:

- `Calibration` was used to configure the network X, specifying seed properties and probabilities of reads and retweets of followers.

Fig. 8 shows the use of FASOW to simulate a WOM campaign, where a developer only needs access to the `Calibration` level. This is because the developer uses `Calibration` to collect and configure already existing modules (*e.g.,* environments and agents). When the modules are configured, an ABM instance is created.

5.2. *Case study 2: Optimal agent seed combination*

5.2.0.1. ***Case Overview.*** This case study replicates the ABM proposed by Araya et al. [14], which investigates optimal combinations of seed agents - hub, leader, and average - for maximizing information diffusion in WOM marketing campaigns conducted over social networks. The replication was conducted using the FASOW architecture to assess its ability to support modular configuration of heterogeneous agent types. The target platform is X, where effective seeding strategies are crucial to real-world campaign outcomes.

5.2.0.2. ***Objectives.*** The goal is to validate whether the FASOW reflective architecture can successfully reproduce the diffusion dynamics observed in the original ABM. Specifically, the aim is to model and simulate different agent roles to identify the combination that yields the highest message reach and persistence over time.

5.2.0.3. ***Data Description.*** Three distinct agent types are modeled:

- **Hub agents**: High connectivity but moderate influence.
- **Opinion leaders**: Fewer connections but greater influence and reach.
- **Average agents**: Median connectivity, representing typical users.

Each agent type contributes differently to the dynamics of message propagation. Their configuration includes parameters such as influence probability, retweet behavior, and follower counts.

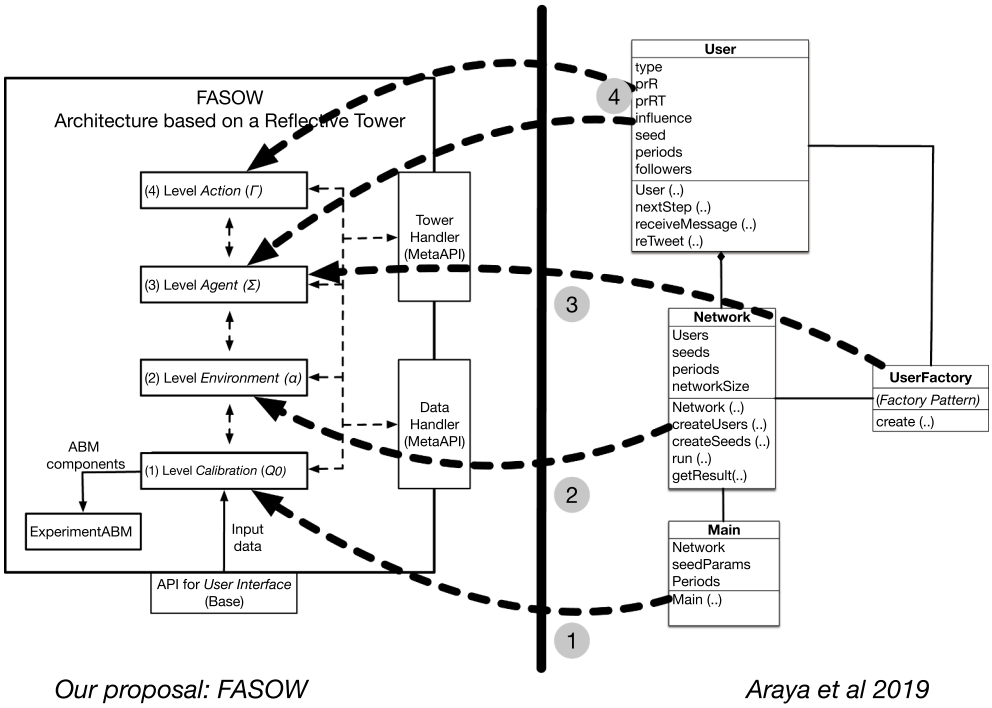


Fig. 9. Replicated components of Araya et al. (2019) in the FASOW Reflective Tower.

5.2.0.4. ***Method Application.*** To replicate the ABM, two key configurations were introduced in the FASOW reflective tower:

- `LevelExtensions` were used to define modules for each new agent type, enabling the architecture to distinguish and simulate their unique behaviors.
- `Calibration` was employed to configure the network X, specifying seed properties and thresholds associated with each agent type.

Fig. 9 illustrates how these components were mapped to FASOW's architectural levels. Each element of the original ABM aligns with specific modules in the reflective tower:

1. The `Calibration` module replicates the initial simulation setup.
2. The `Environment` corresponds to the `Network` class. Although `Network` includes methods associated with `Agent`, the interaction is resolved through calls between the contiguous levels `Environment` and `Agent`.
3. The `UserFactory` class, responsible for generating agents, maps to the `Agent` level.
4. The `User` class is split across two levels: `Agent` and `Action`, since agent behavior is varied and encapsulated in action modules.

5.2.0.5. ***Results.*** The replicated model successfully reproduced the key diffusion dynamics reported in the original study, confirming that agent configuration plays a critical role in determining both message reach and longevity. By introducing distinct agent types with varying influence and connectivity, FASOW not only matched the original outcomes but also allowed for finer control over agent interactions. This replication validated the architecture´s ability to express heterogeneous agent roles and simulate their combined effects on information propagation. Furthermore, the modular implementation reduced development overhead and enabled rapid experimentation with alternative seeding strategies, reinforcing the practical value of the architecture for campaign optimization tasks.

5.3. Case study 3: Effect of message repetition

5.3.0.1. ***Case Overview.*** This case study replicates the ABM proposed by López et al. [13], which investigates the psychological and behavioral effects of message repetition in social networking sites (SNSs). The original study models user engagement with repeated messages through a two-phase behavioral process:

1. **Wear-in Phase:** Initial repetitions enhance understanding and acceptance, gradually fostering positive attitudes.

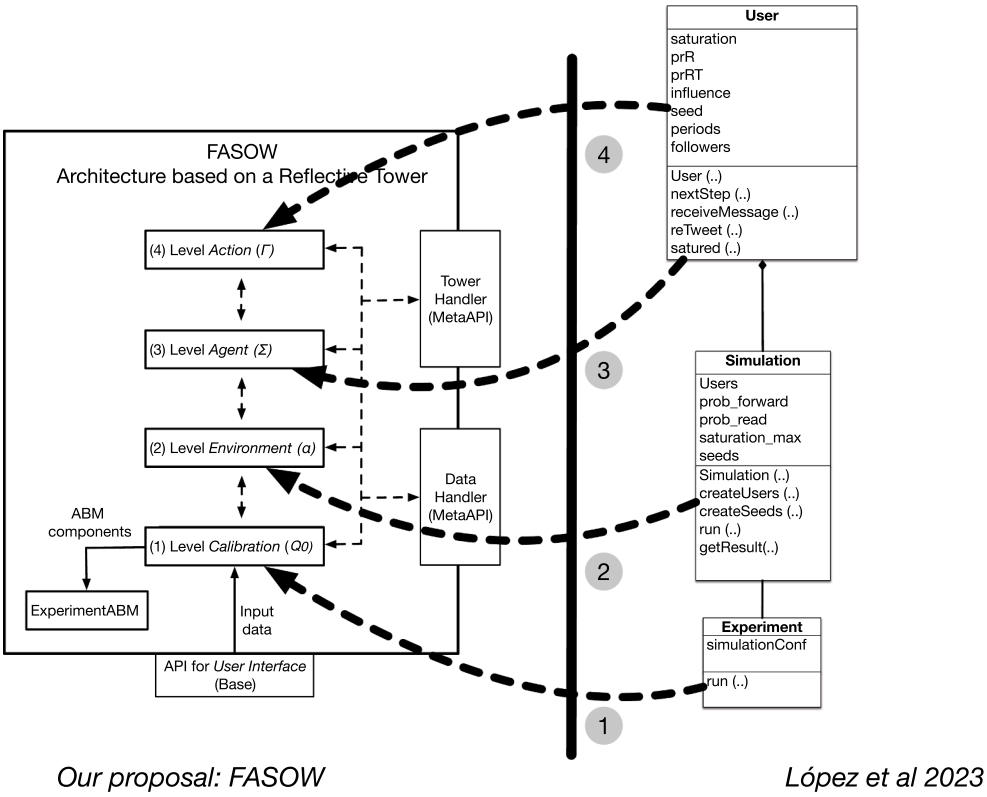


Fig. 10. Replicated components of López et al. (2023) in the FASOW Reflective Tower.

2. **Wear-out Phase:** Continued repetition leads to saturation, triggering negative responses such as disengagement or aversion.

The scenario simulates a company repeatedly posting identical tweets on its X account. The company´s followers (seeds) receive the message, and some agents retweet it. As the repetition increases, user engagement initially grows and then declines due to overexposure.

5.3.0.2. ***Objectives.*** The aim of this case is to validate whether FASOW can capture both phases of the message repetition lifecycle by accurately simulating agent-level saturation dynamics in a realistic social network setting.

5.3.0.3. ***Data Description.*** The original study models a campaign by a winery that sends out repeated tweets. Key parameters include:

- Network setup with 2334 seeded followers.
- Agent states: `not read`, `read`, `retweeted`, and `saturated`.
- Probabilistic rules governing reading and retweeting behaviors.

The model assumes all users belong to the same class, with one boolean flag distinguishing seeds from regular agents.

5.3.0.4. ***Method Application.*** Replicating this study required extending all four levels of the FASOW reflective tower:

- The `Environment` level simulated the `Simulation` class, with network topology and messaging logic.
- The `Agent` level defined agent behaviors, including message engagement and saturation thresholds.
- The `Action` level implemented transitions between states.
- The `Calibration` level configured the action probabilities and initial network parameters.

Fig. 10 illustrates how the architecture used in this case study was mapped to FASOW. Key mappings include:

1. `Calibration` defines the initial simulation parameters.
2. `Environment` emulates the `Simulation` class and uses `Agent` to instantiate users and seeds.
3. & 4. The `User` class is split into `Agent` and `Action` levels to capture saturation behavior, despite having only one user type.

Table 4
Architectural levels used per case study.

Case Study	Levels Used (Extended)	Levels Used (Reused)
N1: WOM-Campaign	None	Calibration, Environment, Agent, Action
N2: MessageRepetition	CalibrationEffectMessageRepetition (Calibration), fasow/fasow/abm (Agent)	None
N3: AgentCombination	fasow/fasow/abm (Agent)	twitter (Environment), fasow/fasow/abm/wom/custom-actions (Action)

Table 5
Coupling metrics for selected modules in the FASOW implementation.

Module	Ca	Ce	I
fasow/fasow/abm/Agent.ts	0	5	1.00
api/app.controller.ts	0	3	1.00
api/app.service.ts	0	1	1.00
main.ts	0	3	1.00
experiments/ExperimentAgentCombinatio/Experiment.ts	0	10	1.00

5.3.0.5. ***Results.*** The replicated simulation accurately reproduced the wear-in and wear-out phases observed in the original study: agent engagement increased during early repetitions, confirming learning and reinforcement effects, and subsequently declined as saturation set in, reflecting message fatigue dynamics. This behavioral progression closely matched the original diffusion curves, demonstrating FASOW ´s ability to simulate non-linear, phase-dependent user responses within a social network environment. The successful replication affirms the architecture´s expressive capacity to capture temporal behavioral shifts driven by cumulative exposure.

5.4. Architecture validation via software metrics

To complement the case study replications and assess the architectural qualities of the FASOW platform, we conducted a quantitative evaluation based on established software engineering metrics. This analysis focuses on modularity, maintainability, and extensibility by computing key module level coupling indicators, such as afferent coupling (Ca), efferent coupling (Ce), and instability (I). By examining both individual TypeScript modules and their organization into architectural levels within the Reflective Tower, we aim to validate FASOW´s ability to support modular development and controlled evolution in agent based modeling scenarios. The following section presents the rationale, metrics, and architectural interpretation that underpin this analysis.

To evaluate the modular structure of a software system, we use the following coupling metrics:

- **Afferent Coupling (Ca)**: Number of modules that depend on a given module. High Ca indicates responsibility and potential resistance to change.
- **Efferent Coupling (Ce)**: Number of modules that a given module depends on. High Ce suggests fragility due to many external dependencies.
- **Instability (I)**: Defined as $I = \frac{Ce}{Ca+Ce}$, instability measures how sensitive a module is to changes in other modules. Values closer to 1 indicate high sensitivity.

To analyze the software modularity and maintainability, we distinguish between:

- **Modules**: Individual TypeScript files (e.g., `main.ts`, `Agent.ts`) for which coupling metrics are computed.
- **Levels**: Directory-level abstractions representing vertical layers of the reflective tower (e.g., `CalibrationEffectMessageRepetition`, `twitter`).

5.4.1. Rationale for selection and interpretation

Each case study either reuses or extends specific levels of the reflective tower (Calibration, Environment, Agent, Action). The selected modules reflect key points of coordination or customization within those levels, either by orchestrating the system´s execution or implementing new behaviors. These modules were chosen because they are central to the implementation logic of each experiment and directly participate in the realization or coordination of architectural level functionality. Their analysis enables us to evaluate how modularity, maintainability, and scalability are preserved as the system evolves through reuse or extension. Table 5.4.1 summarizes the architectural levels used in each case study, while Table 5.4.1 presents the corresponding coupling metrics for the selected modules.

Notice that we distinguish between **modules** (specific TypeScript files, e.g., `Agent.ts`, `Experiment.ts`) and **levels** (directory-based architectural layers, e.g., `CalibrationEffectMessageRepetition`, `twitter`), as both contribute to the system's structure and modular extension strategy.

- **ExampleExperiment**:

  - *Levels Used:* None extended – all reused.
  - *Modules:* `main.ts`, `app.controller.ts`, `app.service.ts`
  - *Rationale:* This experiment reused orchestration logic without introducing new domain-specific functionality. The selected modules handle top-level coordination and serve as an interface layer. Their low coupling values confirm that the architecture supports clean separation of orchestration concerns and enables seamless experiment integration without architectural changes.
- **MessageRepetition**:
  - *Levels Used:* Calibration, Environment, Agent, Action (all extended)
  - *Modules:* `Agent.ts`
  - *Rationale:* The `Agent.ts` module encapsulates new behavior for message propagation. The level directory `CalibrationEffectMessageRepetition` provides a specialized calibration strategy. The metric results confirm that such modular extensions introduce localized dependencies, preserving the integrity of reused components and supporting maintainability as the architecture evolves.
- **AgentCombination**:
  - *Levels Used:* Agent (extended); Environment and Action (reused)
  - *Modules:* `Experiment.ts`
  - *Rationale:* The experiment module orchestrates agent interactions under a reused environment. The setup demonstrates partial level extension and localized behavioral injection. Despite higher efferent coupling due to orchestration logic, the architecture remains scalable and modular as these interactions remain confined to a single coordination module.

5.4.2. Architectural interpretation

The selected modules span distinct roles within the architecture: orchestration (`main.ts`), API exposure (`app.controller.ts`, `app.service.ts`), domain-specific behavior (`Agent.ts`), and experiment-specific coordination (`Experiment.ts`). All exhibit high instability ($I = 1.0$), which might suggest fragility at first glance. However, this is expected and appropriate given their purpose: these modules act as coordination or extension points, designed for use, not reuse, and therefore naturally depend on other components while remaining unreferenced themselves. This pattern reflects a deliberate architectural separation of concerns.

Modularity. The reflective tower decomposes the architecture into four vertical levels; `Calibration`, `Environment`, `Agent`, and `Action`. Each one is responsible for encapsulating specific concerns. Modules are introduced within the appropriate level without cross-layer dependencies, reinforcing encapsulation and reducing coupling. This disciplined modular design facilitates targeted changes without widespread impact.

Maintainability. Each architectural level is independently managed and communicates via well-defined interfaces (e.g., `BaseAPI`, `MetaAPI`). This allows developers to reason about behavior locally, understand modifications in isolation, and minimize unintended side effects. The use of common handlers such as `TowerHandler` and `DataHandler` further promotes centralized control and reduces code duplication.

Extensibility. FASOW supports extensibility through clear conventions for both vertical and horizontal growth:

- **Vertical growth:** New functionality is introduced through level-specific extensions (e.g., `CalibrationEffectMessageRepetition`) without modifying existing infrastructure. This supports safe evolution of the architecture.
- **Per-case reuse and extension:** Case studies demonstrate selective reuse or extension of levels:
  - *AgentCombination* extended the `Agent` level while reusing the existing `Environment` and `Action` layers.
  - *MessageRepetition* implemented full-level extensions across all four layers, using isolated modules and levels.
  - *ExampleExperiment* reused the orchestration logic entirely, relying solely on pre-existing modules without extending any level.

This structured approach confirms that FASOW promotes modular evolution and ensures that growth through new features or experiments can proceed with minimal disruption to existing components.

5.5. Discussion

The case studies presented in this work highlight the flexibility, modularity, and scalability of the FASOW architecture for implementing Agent-Based Models (ABMs) in the context of WOM marketing on Social Network Sites (SNSs). Each case posed distinct modeling requirements and architectural demands, enabling a comprehensive assessment of FASOW's capabilities.

The first case study focuses on highlighting the creation of an ABM using existing modules of each level in the FASOW's reflective tower. For this reason, a developer only needs to utilize the `Calibration` level to configure and create an ABM instance. In the second case study, which focused on optimal seed agent combinations, implementing the model required modifications in only two levels of the reflective tower, `Agent` and `Calibration`. Once these extensions were incorporated, the remaining simulation logic was handled by existing components, requiring no additional changes. This underscores FASOW's capacity to support complex agent roles with minimal reimplementation effort modularly. By contrast, the third case study, centered on modeling message repetition and user saturation, demanded architectural extensions across all four levels of the reflective tower. This more complex replication illustrates the architecture's completeness and flexibility in simulating multi-phase behavioral dynamics. The modular separation of agent states and actions allowed developers to encode behavioral transitions such as wear-in and wear-out while maintaining a clear separation of concerns. These two contrasting scenarios show how FASOW balances simplicity for basic simulations with the expressive power needed for behaviorally rich, domain-specific models.

The software metrics reinforce these architectural claims. High instability values ($I = 1.0$) were consistently observed in modules such as `main.ts`, `app.controller.ts`, `Agent.ts`, and `Experiment.ts`. Rather than indicating architectural fragility, this reflects intentional design, these modules act as orchestration and customization points that depend on many internal elements but are not widely reused themselves. This aligns with the single responsibility principle [74] , promoting a decoupled design that enhances clarity and maintainability.

Moreover, the complete absence of afferent coupling ($Ca = 0$) across these modules confirms that they are loosely coupled and independently replaceable, key attributes for safe, incremental evolution. The distinction between `modules` (individual TypeScript files) and `levels` (directory-based abstractions) further localizes change and minimizes unintended side effects.

The vertical stratification enforced by the reflective tower architecture strengthens modular integrity. By encouraging extensions within clearly scoped layers, Calibration, Environment, Agent, and Action, FASOW avoids cross-level entanglement and supports scalable development through compositional reuse and parallel workstreams.

Although the quantitative metrics provide strong evidence of architectural soundness, it is also important to recognize current limitations. The `Calibration` level currently addresses two concerns, simulation parameter configuration and interface setup for new modules. Combining these roles may reduce clarity and maintainability as the system scales. Future refinements could involve separating these responsibilities to enhance conceptual clarity and simplify maintenance. Acknowledging such design considerations contributes to a more balanced and thorough architectural evaluation.

6. Threats to validity

This section discusses potential threats that may affect the validity of our findings regarding the design, implementation, and evaluation of FASOW. Because our study combines software architecture analysis and the replication of Agent-Based Models (ABMs), we address construct, internal, and external validity threats.

6.1. Construct validity

A central threat concerns the current absence of empirical evidence regarding FASOW's usability and its impact on developer experience. Without user studies or controlled experiments, the effectiveness of the architecture in practical development contexts cannot be empirically established. This threat is partially mitigated by the transparency and inspectability of the contribution: the paper specifies the architecture using C4 container, component, and sequence diagrams, and provides a complete, publicly available implementation (FASOWts) together with a working Web interface. These artefacts allow independent inspection, replication, and experimentation by the community, reducing ambiguity about the behavior and structure of the system. Nevertheless, the absence of formal empirical evaluation remains a limitation, and the paper explicitly acknowledges that claims about usability or ease of extension must be regarded as provisional until supported by systematic studies.

6.2. Internal validity

A threat to internal validity arises from potential ambiguity in how the architecture's reflective mechanisms operate, particularly the interactions among `BaseAPI`, `MetaAPI`, `Level`, and `LevelExtensions`. Misunderstanding these flows could lead to incorrect interpretations of configuration or extension behavior. This threat is partially mitigated through the explicit definition of module responsibilities and their coordination, supported by sequence diagrams that detail the execution and extension protocols. Additionally, the included code example clarifies how reflection is operationalized in practice. While these elements improve interpretability, internal validity ultimately remains limited in the absence of empirical evidence showing that developers consistently understand and apply the architecture as intended.

6.3. External validity

External validity concerns the extent to which FASOW generalizes beyond WOM marketing in Social Network Sites. Although the reflective tower captures common ABM abstractions, the design was evaluated solely within SNS-specific diffusion scenarios. Consequently, applicability to domains like ecological simulations cannot be assumed. We mitigate this by explicitly framing FASOW as a domain-specific architecture; this boundary is an intentional design choice to maximize expressivity and modularity within the WOM context. Furthermore, while the implemented modules are specific, the architectural pattern, separating user configuration from meta-level extension, relies on standard OOP concepts, suggesting the core mechanism is transferable to other complex domains. Nevertheless, without cross-domain comparative studies, we restrict our claims of validity to the reported scope.

7. Conclusions

In this work, we introduced the Flexible Agent Simulator for Open WOM (FASOW), an architecture created to support the principled development of Agent Based Models (ABMs) for Word of Mouth (WOM) marketing on social network sites (SNSs). FASOW is grounded in the idea of the reflective tower, a multi layered structure that organizes modeling tasks into successive levels of abstraction. This structure provides developers with a clear pathway for engaging with complexity, from high level configuration to specialized behavioral extensions, while maintaining coherence across the entire system.

To substantiate the architectural proposal, we provided FASOWts, a complete and publicly accessible implementation in TypeScript. Using this implementation, two case studies demonstrate how the architecture enables the construction and analysis of WOM diffusion models with heterogeneous agent types. The results show that roles such as hub agents and opinion leader agents have a meaningful influence on message persistence and audience engagement. These findings illustrate the value of the architectural approach and offer insights of direct interest to practitioners who design marketing strategies for contemporary SNS environments.

A principal strength of FASOW lies in its modular and extensible design. The layered structure isolates concerns in a way that allows developers and researchers to focus on individual components without losing sight of the broader ABM. New behaviors, agent types, and environment rules can be introduced without disrupting existing components, which supports rapid experimentation and encourages the exploration of diverse modeling scenarios. The case studies show how this flexibility allows researchers to iterate more effectively and obtain deeper understanding of agent interactions and message propagation.

Altogether, FASOW advances the engineering of ABMs for WOM marketing by providing a clear architectural foundation, a functioning implementation, and empirical demonstrations of its utility. The combination of reflective mechanisms with domain specific abstractions establishes a framework that is both flexible and robust, and that offers a promising basis for future research on social media diffusion and consumer behavior in digital environments.

CRediT authorship contribution statement

Paul Leger: Writing - review & editing, Writing - original draft, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Conceptualization; **Daniel San Martín:** Writing - review & editing, Writing - original draft, Investigation; **Diego Nuñez:** Software, Methodology; **Tomás Vélez:** Software, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix A. Extension from the Conference Paper

This paper is an extension of a short paper (8 pages) [20] presented at the International Conference on Enterprise Information Systems (ICEIS 24). This extension includes:

1. **Extended conceptual framework:** The present version of this paper provides an enhanced and detailed conceptual description of the Reflective Tower architecture (6 pages more than the conference version). Specifically, this version clarifies the levels of the Reflective Tower design strategy, along with their corresponding scopes. Additionally, we add *metas API* to access, add, and modify modules at each level. Finally, we describe the FASOW's architecture through a static and dynamic design, where the latter design uses two new sequence diagrams to illustrate how a developer can *a*) create an ABM using existing modules in levels and *b*) add new features to ABMs by adding (isolated) modules to a level that controls a specific concern.
2. **Additional and comprehensive use cases:** While the original conference paper included only a preliminary validation based on a single scenario (the saturation effect on X), this journal submission introduces two additional, substantially different scenarios focusing on optimal seeding strategies for WOM marketing campaigns. Finally, we compare these scenarios using the presentations of FASOW's architecture extensions and metrics.
3. **Expanded and rigorous literature review:** In contrast to the limited related work presented previously, the current paper features an extensive literature review (9 pages more than the conference version), clearly situating our work within the broader landscape of ABM frameworks and reflective architectures. In this version, the comparison deeply highlights the state-of-the-art about scientific works and the state-of-the-practice of current and industrial solutions.
4. **Implementation availability and proof of concept:** Unlike the preliminary demonstration presented previously, the current submission explicitly provides detailed information regarding the availability of our concrete implementation on GitHub and Zenodo (DOI), alongside an accessible online proof of concept. This transparency facilitates reproducibility and practical application by the research and developer communities.
5. **Threats to Validity:** We removed the "Lessons Learned" section to create a more complete section named "Threats to Validity". The original version did not explicitly discuss the threats to the validity of the proposed architecture and its implementations. This version includes a new dedicated section (Section 6) that systematically addresses these threats across three dimensions: Construct Validity, Internal Validity, and External Validity. This addition provides a rigorous discussion on the limitations of the current proposal, such as the lack of empirical user studies and the generalization of the architecture beyond WOM marketing, along with the mitigation strategies adopted.

References

[1] C. Newberry, 42 Facebook Statistics Marketers Need to Know in 2023, 16, 2023. https://blog.hootsuite.com/facebook-statistics.
[2] Omnicore, Twitter by the Numbers: Stats, Demographics & Fun Facts, 16, 2023. https://www.omnicoreagency.com/twitter-statistics.
[3] A.M. Kaplan, M. Haenlein, Users of the World, Unite! The Challenges and Opportunities of Social Media 53 (2010) 59–68.
[4] T. Hennig-Thurau, K.P. Gwinner, G. Walsh, D.D. Gremler, Electronic word-of-Mouth via consumer-opinion platforms: what motivates consumers to articulate themselves on the internet?, J. Interact. Market. 18 (2004) 38–52.

[5] R.V. Kozinets, K.D. Valck, A.C. Wojnicki, S.J. Wilner, Networked narratives: understanding word-of-Mouth marketing in online communities, J. Mark. 74 (2010) 71–89.
[6] P. Gupta, J. Harris, How e-WOM recommendations influence product consideration and quality of choice: a motivation to process information perspective, J. Bus. Res. 63 (2010) 1041–1049.
[7] T. Hennig-Thurau, C. Wiertz, F. Feldhaus, Does twitter matter? the impact of microblogging word of mouth on consumers adoption of new movies, J. Acad. Market. Sci. 43 (2015) 375–394.
[8] B.J. Jansen, M. Zhang, K. Sobel, A. Chowdury, T. Power, Tweets as electronic word of mouth, J. Am. Soc. Inf. Sci. Technol. 60 (2009) 2169–2188.
[9] W. Shen, J. Hahn, Impact of Online Word-of-Mouth on the Market for Consumer Goods: The Interplay Between Adoption Rate, Product Market Life and Market Size, 2008. Working paper.
[10] W. Rand, R.T. Rust, Agent-Based modeling in marketing: guidelines for rigor, Int. J. Res. Market. 28 (2011) 181–193.
[11] V. Grimm, U. Berger, F. Bastiansen, S. Eliassen, V. Ginot, J. Giske, J. Goss-Custard, T. Grand, S.K. Heinz, G. Huse, A. Huth, J.U. Jepsen, C.J. rgensen, W.M. Mooij, B. Müller, G. Pe'er, C. Piou, S.F. Railsback, A.M. Robbins, M.M. Robbins, E. Rossmanith, N. Rüger, E. Strand, S. Souissi, R.A. Stillman, R. Vabø, U. Visser, D.L. Deangelis, A standard protocol for describing individual-Based and agent-Based models, Ecol. Modell. 198 (2006) 115–126.
[12] O. Terán, P. Leger, M. López, Modeling and simulating chinese cross-border e-Commerce: an agent-Based simulation approach, J. Simul. 17 (2022) 1–18.
[13] M. López, C. Hidalgo-Alcázar, P. Leger, The effect of message repetition on information diffusion on twitter: an agent-Based approach, 2023, pp. 150–169.
[14] N. Araya, P. Leger, M. López, Whom do i choose to diffuse information ontwitter? an agent-Based model approach, IEEE Lat. Am. Trans. 17 (2019) 677–683.
[15] K. Nygaard, O.-J. Dahl, The Development of the SIMULA Languages, in: History of programming languages, 1978.
[16] U. Wilensky, Netlogo: A Multi-Agent Programmable Modeling Environment, 16, 2023. https://ccl.northwestern.edu/netlogo.
[17] M.J. North, N.T. Collier, J. Ozik, E.R. Tatara, C.M. Macal, M. Bragen, P. Sydelko, Complex Adaptive Systems Modeling with Repast Simphony 1 (2013).
[18] S. Luke, C. Cioffi-Revilla, L. Panait, K. Sullivan, G. Balan, Mason, A Multiagent Simulation Environment 81 (2005) 517–527.
[19] M.H. Ibrahim, Reflection in object-Oriented programming, Int. J. Artif. Intell. Tools 1 (1992) 117–136.
[20] D.N. nez, T. Vélez, P. Leger, D.S. Martín, A reflective architecture for agent-Based models applied to social network sites, in: International Conference on Enterprise Information Systems (ICEIS), Angers, France, 2024, pp. 981–988.
[21] FASOW: Flexible agent simulator for open WOM for typescript (source code), 2025. https://github.com/pragmaticslaboratory/fasow.
[22] FASOWts: A running proof-of-concept of FASOW, 2025. https://sasow-org.github.io/fasow-monorepo.
[23] M. López, M. Sicilia, P. How, WOM Marketing contributes to new product adoption: testing competitive communication strategies, Eur. J. Mark. 47 (2013) 1089–1114.
[24] S. Sen, D. Lerman, Why are you telling me this? an examination into negative consumer reviews on the web, J. Inter. Market. 21 (2007) 76–94.
[25] C.M. Cheung, D.R. Thadani, The impact of electronic word-Of-Mouth communication: a literature analysis and integrative model, Decis. Support Syst. 54 (2012) 461–470.
[26] T. Zhang, D. Zhang, Agent-based simulation of consumer purchase decision-Making and the decoy effect, J. Bus. Res. 60 (2007) 912–922.
[27] B. Libai, E. Muller, R. Peres, Decomposing the value of word-of-mouth seeding programs: acceleration versus expansion, J. Market. Res. 50 (2013) 161–176.
[28] O. Hinz, B. Skiera, C. Barrot, J.U. Becker, Seeding strategies for viral marketing: an empirical comparison, J. Mark. 75 (2011) 55–71.
[29] T. Araujo, P. Neijens, R. Vliegenthart, Getting the word out on twitter: the role of influentials, information brokers and strong ties in building word-Of-Mouth for brands, Int. J. Adver. 36 (2017) 496–513.
[30] L.D. Chen, M.L. Gillenson, D.L. Sherrell, Enticing online consumers: an extended technology acceptance perspective, Inf. Manag. 39 (2002) 705–719.
[31] J.V. Neumann, A.W. Burks, Theory of Self-Reproducing Automata, University of Illinois Press, USA, USA, 1966.
[32] J. Goldenberg, B. Libai, E. Muller, Talk of the Network: A Complex Systems Look at the Underlying Process of Word-of-Mouth 12 (2001) 211–223.
[33] M. Chica, W. Rand, Building agent-Based decision support systems for word-of-Mouth programs: a freemium application, J. Market. Res. 54 (2017).
[34] P. Leger, M. López, C. Hildago-Alcázar, H. Fukuda, An open agent-Based model to simulate the effect of WOM marketing campaigns, EAI Endorsed Trans. Smart Cities 1 (2016).
[35] V. Pizarro, P. Leger, C. Hidalgo-Alcázar, I. Figueroa, Routeplanner, An agent-Based model simulation for suggesting preference-Based routes in spain, J. Simul. 17 (2022) 444–461.
[36] M. López, M. Sicilia, Determinants of E-WOM influence: the role of Consumers' internet experience, J. Theor. Appl. Electron. Commerce Res. 9 (2014) 28–43.
[37] N. Mohammad, On the contributions and the validation of an agent-Based simulation model of innovation diffusion, Eur. J. Mark. 50 (2016) 647–657.
[38] M. Amini, T. Wakolbinger, M. Racer, M.G. Nejad, Alternative supply chain production-Sales policies for new product diffusion: an agent-Based modeling and simulation approach, Eur. J. Oper. Res. 216 (2012) 301–311.
[39] S.A. Delre, W. Jager, T.H. Bijmolt, M.A. Janssen, Will it spread or not? the effects of social influences and network topology on innovation diffusion, J. Product Innovat. Manag. 27 (2010) 267–282.
[40] B.C. Smith, Reflection and semantics in lisp, in: Proceedings of the 11Th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages, the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languagesUtah, USA, 1984, pp. 23–35.
[41] I.R. Forman, Java Reflection in Action, USA, Manning Publications, 2005.
[42] B. Foote, R.E. Johnson, Reflective facilities in smalltalk-80, ACM Sigplan Notices 24 (1989) 327–335.
[43] Nodejs, A JavaScript runtime built for the server side, 2025. https://nodejs.org.
[44] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design patterns: abstraction and reuse of object-oriented design, in: European Conference on Object-Oriented Programming, Springer, 1993, pp. 406–431.
[45] A. Marković, S. Čavoški, A. Novović, Analysis of interactions of key stakeholders on B2C e-Markets - Agent based modelling and simulation approach, Organizacija 49 (2016) 138–149.
[46] M. Chica, W. Rand, Building agent-Based decision support systems for word-of-Mouth programs: a freemium application, J. Market. Res. 54 (2017) 752–767.
[47] M. Liao, L. Qi, J. Zhang, The dynamic evolution mechanism of heterogeneous OWOM-An improved viral marketing model, Information 11 (2020).
[48] Y. Zhang, J. Gao, A. Bilgihan, M. Lorenz, A holistic assessment of ewom management effectiveness with agent-Based modeling, Int. J. Contemp. Hospital. Manag. 35 (2023) 785–827.
[49] O. Terán, P. Leger, M. López, Factors that drive market share and the oligopolistic character of cross-border B2C ecommerce: an agent-based scenario analysis approach, 2024. To appear.
[50] M. Felleisen, On the expressive power of programming languages, Sci. Comput. Program. 17 (1991) 35–75.
[51] R. Harper, Practical Foundations for Programming Languages, 2, Cambridge University Press, 2016. Nd ed.
[52] S.F. Railsback, V. Grimm, Agent-Based and Individual-Based Modeling: A Practical Introduction, Princeton University Press, 2012.
[53] N. Ford, R. Parsons, P. Kua, Building Evolutionary Architectures: Support Constant Change, O'Reilly Media, 2020.
[54] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, 3, Addison-Wesley, 2012. Rd ed.
[55] C.M. Macal, M.J. North, Tutorial on agent-based modeling and simulation, in: Proceedings of the Winter Simulation Conference, the Winter Simulation Conference, 2005, pp. 2–15. https://doi.org/10.1109/WSC.2005.1574234
[56] E. Ecolab, 16, 2010. https://ecolab.sourceforge.net.
[57] G. Ciatto, A. Omicini, S. Mariani, Tucson, 16, 2015. https://apice.unibo.it/xwiki/bin/view/TuCSoN.
[58] P. Chapron, J. El-Gemayel, M. Mailliard, L. Sanchez-Arias, Soclab, 16, 2004. https://soclabproject.wordpress.com.
[59] P. Andreade, T. Carneiro, Terrame, 16, 2006. http://www.terrame.org/doku.php?id=start.
[60] I. Nunes, BDI4Jade, 16, 2014. https://github.com/ingridnunes/bdi4jade.
[61] J.D. Mooij, Sim2APL, 16, 2021. https://bitbucket.org/goldenagents/sim2apl/src/master.
[62] U. S, DGtal Aqua Lab, ReMobidyce, 16, 2024. https://github.com/ReMobidyc/ReMobidyc.
[63] T.A. Company, Anlylogic, 16, 2000. https://www.anylogic.com/.

[64] A. Extendsim, Inc, 16, 2024. https://extendsim.com.
[65] I. Flexsim, Software Products, FlexSim, 16, 2008. https://www.flexsim.com.
[66] O. Boissier, R.H. Bordini, J.F. Hübner, A. Ricci, Jacamo, 16, 2023. https://jacamo-lang.github.io.
[67] S. Team, Simona, 16, 2024. https://simona.ie3.e-technik.tu-dortmund.de.
[68] P. Richmond, G. Flame, 16, 2023. https://flamegpu.com.
[69] A. Horni, K. Nagel, K.W. Axhausen, The Multi-Agent Transport Simulation MATSim, Ubiquity Press, 2016. https://matsim.org.doi:10.5334/baw. https://doi.org/10.5334/baw
[70] A.N. Laboratory, R. Symphony, 16, 2013. https://repast.github.io.
[71] Symudine, Enterpr. Simul. Softw. 16 (2016). https://simudyne.com.
[72] U. Wilensky, W. Rand, An Introduction to Agent-Based Modeling: Modeling Natural, Social, and Engineered Complex Systems with NetLogo, The, MIT Press, 2015.
[73] J. Zhang, D.T. Robinson, Replication of an agent-based model using the replication standard, Environ. Modell. Softw. 139 (2021) 105016.
[74] R.C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship, USA, Prentice Hall PTR, 2008. 1 ed.